

EPFL CIVIL-127, Lab 7 solution

lab7.py

```
from typing import Iterable

def count_patterns(i: Iterable[int]) -> tuple[int, int, int]:
    """
    Counts patterns per lab 7.1 assignment
    """
    three_consecutive = 0
    all_different = 0
    alternating = 0
    for n in i:
        if has_three_consecutive(str(n)):
            three_consecutive += 1
        if are_all_different(str(n)):
            all_different += 1
        if is_alternating(str(n)):
            alternating += 1
    return (three_consecutive, all_different, alternating)

def has_three_consecutive(s: str) -> bool:
    """
    Returns true if s contains any 3 consecutive characters
    """
    for i in range(len(s)-2):
        if s[i] == s[i+1] and s[i] == s[i+2]:
            return True
    return False

def are_all_different(s: str) -> bool:
    """
    Returns true if s contains all different characters
    """
    return len(s) == len(set(s))

def is_alternating(s: str) -> bool:
    """
    Returns true if s contains characters which alternate between > and <
    """
```

Note: the problem description doesn't specify how to handle strings of length 0, 1, and 2. We make the arbitrary decision to consider all strings of length 0, 1 and 2 as not alternating.

```
"""
if len(s) < 3:
    return False
for i in range(0, len(s)-2):
    if s[i] >= s[i+1] and s[i+1] >= s[i+2]:
        return False
    if s[i] <= s[i+1] and s[i+1] <= s[i+2]:
        return False
return True

def my_mod_exp(base: int, exp: int, mod: int) -> int:
    """
    Returns (base ** exp) % mod, i.e. the exact same behavior as Python's
    built-in pow() function. See lab 7.2 assignment.

    Implemented using recursion.
    """
    if exp == 0:
        return 1
    elif exp % 2 == 0:
        n = my_mod_exp(base, exp//2, mod)
        return (n * n) % mod
    else:
        n = my_mod_exp(base, exp-1, mod)
        return (base * n) % mod

def last_10_digits(i: Iterable[int]) -> int:
    """
    Returns the last 10 digits of raising each element of the iterable (in
    reverse order) to the power of the next elements. I.e. if i has four
    elements, a, b, c, and d, it returns the last 10 digits of:
    d ** (c ** (b ** a))

    The code calls my_mod_exp for each element. There are more efficient ways
    to calculate the final result (involving Carmichael, Euler's totient, or
    Chinese Remainder Theorem), but simply calling my_mod_exp() is fast enough.
    This is all quite amazing since the original number is gigantic!
    """
    r = 1
    for n in i:
        r = my_mod_exp(n, r, 10_000_000_000)
```

```
return r
```

lab7_test.py

```
import unittest

import lab7

class TestLab7(unittest.TestCase):

    def test_count_patterns(self):
        a = lab7.count_patterns(range(1000, 10000))
        self.assertEqual(a, (171, 4536, 3105))

    def test_has_three_consecutive(self):
        self.assertTrue(lab7.has_three_consecutive("122234"))
        self.assertFalse(lab7.has_three_consecutive("9322"))

    def test_are_all_different(self):
        self.assertTrue(lab7.are_all_different("92103"))
        self.assertFalse(lab7.are_all_different("71479"))

    def test_is_alternating(self):
        self.assertTrue(lab7.is_alternating("285609"))
        self.assertTrue(lab7.is_alternating("638342"))
        self.assertFalse(lab7.is_alternating("842305"))
        self.assertFalse(lab7.is_alternating("285600"))

    def test_my_mod_exp(self):
        a = lab7.my_mod_exp(29, 39, 541)
        self.assertEqual(a, (29 ** 39) % 541)

    def test_last_10_digits(self):
        a = lab7.last_10_digits([2, 3, 4])
        self.assertEqual(a, 4**(3**2))

        a = lab7.last_10_digits([2, 3, 4, 5, 6, 7, 8, 9])
        self.assertEqual(a, 8779806721)

if __name__ == "__main__":
    unittest.main()
```